

Clasificación Automática de Ofertas de Empleo y Agente Evaluador de Hojas de Vida

Informe Técnico Completo

Mateo Villada Higueta

Samuel Herrera

Santiago Ramirez

Proyecto Final IA

23 de mayo de 2026

Índice

1. Planteamiento del problema	2
2. Dataset y construcción de etiquetas	2
2.1. Agrupamiento a macro-categorías	2
2.2. Decisiones metodológicas de limpieza	2
3. Análisis exploratorio	2
4. Metodología de entrenamiento	3
4.1. Preprocesamiento	3
4.2. Vectorización	3
4.3. Modelos evaluados	3
5. Resultados del clasificador	3
6. Conclusiones del clasificador y trabajo futuro	4
7. Resumen del agente evaluador de hojas de vida	5
8. Descripción detallada del agente	5
8.1. Patrón ReAct	5
8.2. Stack tecnológico	5
8.3. Herramienta definida	6
8.4. Prompt engineering	6
8.5. Evaluación estandarizada	6
8.6. Análisis de fallos	7
8.7. Decisiones de diseño documentadas	7
9. Conclusiones generales	8

1. Planteamiento del problema

El mercado laboral digital genera diariamente miles de ofertas de empleo en plataformas como LinkedIn, Indeed y Glassdoor. La indexación manual de estas vacantes es inconsistente y dificulta el análisis automatizado de la demanda por sector. Este proyecto implementa un pipeline de clasificación supervisada que asigna automáticamente a cada vacante una categoría industrial, sentando la base para un segundo componente que evalúa la compatibilidad de hojas de vida frente a las vacantes clasificadas mediante un agente basado en LLM.

Este informe documenta el trabajo completado en: construcción del dataset etiquetado, análisis exploratorio, decisiones metodológicas, entrenamiento y evaluación del clasificador; y en una segunda parte, la implementación del agente evaluador de CVs.

2. Dataset y construcción de etiquetas

Se utilizó el dataset público *LinkedIn Job Postings* de Kaggle ([arshkon/linkedin-job-postings](#)), que contiene 123.849 vacantes con descripciones, títulos y metadatos asociados. La etiqueta de industria no está presente en la tabla principal de vacantes (`postings.csv`), por lo que se construyó mediante el cruce con dos tablas auxiliares: `job_industries.csv` (relación vacante-industria) y `industries.csv` (catálogo de industrias). El resultado fue un dataset etiquetado de 122.308 filas con la industria granular de LinkedIn como variable objetivo.

2.1. Agrupamiento a macro-categorías

LinkedIn maneja más de 140 industrias granulares (*Software Development, Hospitals and Health Care, Investment Banking*, etc.), lo que generaría un problema de clasificación de altísima dimensionalidad con clases extremadamente desbalanceadas. Se realizó un agrupamiento manual a **14 macro-categorías** basadas en afinidad de perfiles de skills más que en sector económico puro, lo que resulta más útil para el sistema de comparación CV-vacante. El mapeo cubrió el 90,7 % de las filas originales; el 9,3 % restante (industrias residuales de baja frecuencia) se desartó.

2.2. Decisiones metodológicas de limpieza

Durante el análisis exploratorio se identificaron tres problemas de calidad que motivaron filtros adicionales sobre el dataset:

1. **Exclusión de la categoría HR & Staffing.** El análisis de tokens más frecuentes reveló que las vacantes etiquetadas bajo esta categoría no corresponden a perfiles de recursos humanos sino a agencias de *staffing* publicando para terceros sectores. Sus tokens dominantes (`nurse`, `rn`, `careerstaff`, `patient`, `nursing`) son idénticos a los de Healthcare. Mantener esta categoría introducía ruido de etiquetado al confundir la industria de la empresa con el cargo real ofertado. Se eliminaron 8.685 filas (7,8 %).
2. **Duplicados exactos.** Se identificaron 14.156 vacantes con descripción idéntica (re-publicaciones), las cuales se eliminaron antes del split para evitar fugas entre los conjuntos de entrenamiento y prueba.
3. **Descripciones triviales.** Se descartaron 177 vacantes con menos de 30 palabras de descripción por aportar señal insuficiente.

El dataset final consta de **87.910 vacantes** distribuidas en **13 macro-categorías**, con un desbalance manejable de aproximadamente 14:1 entre la clase mayoritaria (Healthcare, 23,4 %) y la minoritaria (Consulting & Professional Services, 1,7 %).

3. Análisis exploratorio

El EDA se enfocó en tres dimensiones críticas para el modelado posterior: distribución de clases, longitud de las descripciones y separabilidad léxica entre categorías.

Distribucion de clases. Healthcare (20.571), IT & Software (15.641) y Manufacturing & Engineering (10.904) concentran el 53 % del dataset. Las clases minoritarias (Marketing, Non-profit, Transportation y Consulting) tienen entre 1.500 y 2.400 muestras cada una, volumen suficiente para entrenar con `class_weight` balanceado.

Longitud de descripciones. La mediana por categoria oscila entre 300 y 600 palabras, con distribuciones uniformes entre clases. Esto descarta que diferencias de longitud expliquen artificialmente la clasificacion.

Separabilidad lexica. Mediante TF-IDF exploratorio se identificaron los tokens mas caracteristicos de cada clase, confirmando vocabularios altamente distintivos: Healthcare (`patient`, `nursing`, `clinical`, `hospital`), Education (`students`, `teacher`, `faculty`), Legal (`litigation`, `attorney`, `law`, `firm`), Hospitality (`hotel`, `guest`, `hilton`, `hyatt`). Las categorias mas genericas lexicamente son Consulting (`project`, `clients`, `business`) e IT & Software (`experience`, `data`, `technical`), lo que anticipo las confusiones observadas en la evaluacion.

4. Metodologia de entrenamiento

4.1. Preprocesamiento

Se concateno titulo y descripcion en un unico campo de texto. La limpieza fue ligera: remocion de etiquetas HTML, URLs y caracteres no alfanumericos. Se omitieron deliberadamente stemming y lematizacion tras verificar que la separabilidad lexica entre clases era alta sin estos pasos, optando por priorizar velocidad de iteracion.

4.2. Vectorizacion

Se aplico TF-IDF con los siguientes hiperparametros:

- `max_features=15000`, `ngram_range=(1,2)` (unigramas y bigramas)
- `min_df=5`, `max_df=0.9`, `stop_words='english'`
- `sublinear_tf=True` (escalado logaritmico de frecuencias)

4.3. Modelos evaluados

Se entrenaron dos modelos sobre un split estratificado 80/20 (70.328 train, 17.582 test):

- **Logistic Regression** (`liblinear`, `C=1.0`, `class_weight='balanced'`, `max_iter=200`).
- **LightGBM** (`n_estimators=200`, `learning_rate=0.1`, `num_leaves=31`, `class_weight='balanced'`).

El entrenamiento de LightGBM no convergio en tiempo razonable sobre la infraestructura disponible (Colab gratuito, CPU), debido al costo de entrenar internamente un arbol por clase por iteracion (200 estimadores x 13 clases = 2.600 arboles). Tras multiples intentos con configuraciones reducidas, se adopto **Logistic Regression como modelo final**. Esta decision es metodologicamente valida: en clasificacion de texto multiclase con TF-IDF, los modelos lineales son competitivos con *gradient boosting*, especialmente cuando el vocabulario distintivo por clase es claro.

Adicionalmente, se exploró inicialmente el solver `saga`, que se descarto tras no converger en mas de 20 minutos; `liblinear` resulto funcionalmente equivalente para este caso multiclase y convergio en aproximadamente 2 minutos.

5. Resultados del clasificador

La Tabla 1 resume las metricas globales del modelo final sobre el conjunto de prueba.

Los tres valores superan los umbrales minimos aceptables definidos en la guia del proyecto (Accuracy > 0,80, F1-macro > 0,75). La Tabla 2 detalla el desempeno por categoria.

Cuadro 1: Metricas globales del modelo final (Logistic Regression).

Metrica	Valor
Accuracy	0,8300
F1-macro	0,7985
F1-weighted	0,8314

Cuadro 2: F1-score por clase, ordenado de mejor a peor.

Categoria	Precision	Recall	F1
Healthcare	0,951	0,895	0,922
Education	0,879	0,864	0,871
Hospitality & Travel	0,850	0,882	0,866
Retail & Consumer	0,883	0,824	0,853
Legal & Government	0,817	0,824	0,820
Finance	0,822	0,813	0,817
Construction & Real Estate	0,801	0,801	0,801
IT & Software	0,776	0,816	0,795
Manufacturing & Engineering	0,779	0,796	0,787
Transportation & Logistics	0,724	0,788	0,754
Non-profit & Social	0,737	0,760	0,748
Marketing & Media	0,640	0,799	0,711
Consulting & Professional Services	0,635	0,633	0,634

Analisis cualitativo. Doce de las trece categorias superan el umbral de $F1 > 0,65$ establecido en la guia. La unica clase por debajo es Consulting & Professional Services (0,634), lo cual era predecible desde el EDA: las consultoras publican vacantes tecnicas (Data Engineer, Financial Analyst, Software Developer) que pertenecen funcionalmente a otras categorias. La matriz de confusion normalizada confirma las confusiones principales:

- Consulting se confunde principalmente con IT & Software (16 %) y Finance (5 %).
- Finance pierde un 11 % de instancias hacia IT & Software, reflejando el alto componente tecnico de la banca moderna.
- Marketing & Media confunde 9 % hacia IT & Software, asociado a perfiles de marketing digital.

Estas confusiones no son fallas del modelo sino reflejo de la mezcla real de perfiles en el mercado laboral, donde la industria de la empresa empleadora no siempre coincide con el cargo ofertado.

6. Conclusiones del clasificador y trabajo futuro

El pipeline implementado clasifica vacantes en 13 macro-categorias industriales con metricas que superan los umbrales academicos definidos (F1-macro 0,799, Accuracy 0,830). Las decisiones metodologicas clave —agrupamiento de industrias granulares, exclusion de HR & Staffing por contaminacion de etiquetas, y eliminacion de duplicados— resultaron determinantes para la calidad del modelo. Los artefactos persistidos (vectorizador, clasificador y *label encoder*) estan listos para integrarse con el agente LLM.

Como trabajo futuro inmediato se contempla: (i) reintento de LightGBM con muestreo o reduccion de features sobre infraestructura con mayor capacidad de computo; (ii) evaluacion de DistilBERT como modelo alternativo en GPU; (iii) re-etiquetado heuristico de la categoria HR & Staffing por contenido del cargo en lugar de industria de la empresa, para recuperar las 8.685 vacantes descartadas.

7. Resumen del agente evaluador de hojas de vida

El segundo componente del sistema es un agente de IA que evalúa automáticamente la compatibilidad entre una hoja de vida en PDF y una oferta de empleo. Implementado con el patrón *ReAct* (LangGraph + Llama-3.1-8b-instant via Groq API), el agente lee el CV, razona sobre la compatibilidad y devuelve un JSON estructurado con: score de 0 a 100, resumen del perfil, fortalezas, brechas y tres recomendaciones accionables.

Motivación. Evaluar manualmente la compatibilidad de un candidato toma entre 10 y 20 minutos por CV. El agente reduce este tiempo a menos de 20 segundos.

Arquitectura. El ciclo *ReAct* ejecuta al menos una iteración: el LLM invoca la herramienta `read_cv_pdf` para extraer el texto del PDF, y luego produce la evaluación JSON como respuesta final. La temperatura se fija en 0.2 para obtener scores determinísticos y comparables; valores más altos producían variabilidad de hasta ± 15 puntos para el mismo input.

Evaluación. Se probaron 15 casos estandarizados (5 de nivel alto, 5 medio, 5 bajo) usando CVs sintéticos generados con `fpdf2`. El agente alcanzó una **tasa de acierto del 86.7%** (13/15 casos correctos) con una latencia promedio de 15.6 segundos. Los dos fallos corresponden a perfiles con habilidades transferibles: un profesor universitario cuyo score cayó justo por debajo del umbral de nivel medio, y un chef ejecutivo al que el agente asignó compatibilidad parcial por sus habilidades de gestión, sin penalizar suficientemente la brecha de dominio técnico.

Decisiones de diseño destacadas. Se usó LangGraph sobre LangChain clásico por ser la API actualmente mantenida; el prompt del sistema se redactó en inglés por mayor fidelidad en el seguimiento de instrucciones de herramientas; y el parseo del output se realiza con expresión regular para manejar texto introductorio adicional que el LLM puede incluir antes del JSON.

8. Descripción detallada del agente

8.1. Patrón ReAct

El agente implementa el patrón *ReAct* (Reasoning and Acting), propuesto por Yao et al. (2022), que intercala pasos de razonamiento con llamadas a herramientas externas. El ciclo de ejecución es el siguiente:

1. **Thought:** el LLM analiza la instrucción y razona sobre qué acción debe tomar primero.
2. **Action:** el LLM invoca una herramienta disponible con los parámetros que considera necesarios.
3. **Observation:** el sistema ejecuta la herramienta y devuelve el resultado al LLM.
4. El ciclo se repite hasta que el LLM determina que tiene suficiente información para producir la respuesta final.

8.2. Stack tecnológico

Cuadro 3: Stack del agente evaluador

Componente	Tecnología	Version
Framework de agente	LangGraph (<code>create_react_agent</code>)	0.2.x
Modelo de lenguaje	Llama-3.1-8b-instant via Groq API	—
Temperatura	0.2	—
Extracción de PDF	pdfplumber	0.11.x
Validación de output	Pydantic (<code>BaseModel</code>)	2.x
Interfaz	Streamlit	1.x

8.3. Herramienta definida

```
1 @tool
2 def read_cv_pdf(pdf_path: str) -> str:
3     """
4     Lee el texto completo de una hoja de vida en PDF dado su path.
5     Usar cuando se necesite evaluar el perfil del candidato.
6     """
7     text = ""
8     with pdfplumber.open(pdf_path) as pdf:
9         for page in pdf.pages:
10             page_text = page.extract_text()
11             if page_text:
12                 text += page_text + "\n"
13     if len(text.strip()) < 100:
14         raise CVReadError(
15             "El PDF esta vacio o es una imagen escaneada."
16         )
17     return text
```

Listing 1: Definicion de la herramienta read_cv_pdf

8.4. Prompt engineering

El prompt del agente incluye reglas de scoring explicitas para forzar consistencia entre evaluaciones:

Cuadro 4: Reglas de scoring definidas en el prompt

Condicion	Rango de score	Nivel
Mas del 90 % de requisitos cumplidos con deseables	90–100	Alto
75–89 % de requisitos clave cumplidos	70–89	Alto
50–74 % de requisitos clave cumplidos	40–69	Medio
Menos del 50 % de requisitos clave cumplidos	0–39	Bajo

8.5. Evaluacion estandarizada

Se disenaron 15 casos de prueba con CVs sinteticos en tres niveles de compatibilidad:

Cuadro 5: Resultados de la evaluacion estandarizada (15 casos)

Caso	Candidato	Esperado	Score	Predicho	Acierto	Latencia (s)
1	Senior Dev	Alto	88	Alto	Si	14.2
2	Financial CFA	Alto	82	Alto	Si	13.8
3	Nurse UCI	Alto	91	Alto	Si	16.1
4	Marketing Mgr	Alto	85	Alto	Si	15.3
5	Civil PMP	Alto	87	Alto	Si	14.9
6	Junior Analyst	Medio	42	Medio	Si	17.2
7	Profesor	Medio	38	Bajo	No	15.6
8	Contador SAP	Medio	60	Medio	Si	16.8
9	UX Designer	Medio	45	Medio	Si	14.7
10	Logistica	Medio	41	Medio	Si	15.9
11	Chef	Bajo	60	Medio	No	18.3
12	Historia Grad	Bajo	18	Bajo	Si	13.4
13	Retail Vendor	Bajo	12	Bajo	Si	14.1
14	Medico General	Bajo	22	Bajo	Si	15.7
15	Estudiante	Bajo	15	Bajo	Si	13.9
Tasa de acierto					13/15 (86.7 %)	15.6 prom.

Cuadro 6: Resultados agregados por nivel

Nivel	Casos	Acertados	Tasa
Alto	5	5	100 %
Medio	5	4	80 %
Bajo	5	4	80 %
Total	15	13	86.7 %

8.6. Analisis de fallos

Caso 7 — Profesor universitario (esperado: medio, predicho: bajo). El agente asigno un score de 38, justo por debajo del umbral de nivel medio. El perfil incluia conocimientos basicos de Python y estadistica, pero la vacante requería Spark, Airflow y AWS. El score cayo en el rango bajo en lugar del medio esperado, lo que sugiere que el umbral de 40 puntos puede ser demasiado alto para perfiles con formacion academica relevante pero sin experiencia industrial directa.

Caso 11 — Chef ejecutivo (esperado: bajo, predicho: medio). El agente asigno un score de 60, identificando como fortalezas la gestion de personal y liderazgo de equipos. El agente no aplico la penalizacion de dominio cruzado con suficiente peso; un reclutador humano descartaria este perfil de inmediato.

8.7. Decisiones de diseno documentadas

1. **LangGraph sobre LangChain clasico.** `create_react_agent` de LangGraph es la API actualmente mantenida y mas estable para agentes ReAct.
2. **Temperatura 0.2.** Reduce la variabilidad del score a rangos comparables; valores mas altos producian diferencias de ± 15 puntos para el mismo input.
3. **Una sola herramienta.** Se descarto agregar herramientas adicionales para evitar loops de razonamiento innecesarios.

4. **CVs sinteticos.** Generados con `fpdf2` para controlar el nivel de compatibilidad esperado y garantizar reproducibilidad sin usar datos personales.
5. **Prompt en ingles.** Llama-3 sigue instrucciones de herramientas con mayor fidelidad en ingles.
6. **Archivo temporal para el PDF.** Streamlit no provee paths reales; se uso `tempfile.NamedTemporaryFile` con un bloque `finally` para limpieza.
7. **Parseo con expresion regular.** El output del agente puede incluir texto introductorio; se extrae el bloque JSON con `re.search(r'{}'.*', raw, re.DOTALL)`.
8. **Latencia como metrica operacional.** La latencia promedio de 15.6 s es aceptable frente a los 10–20 minutos del proceso manual.

9. Conclusiones generales

El sistema completo integra un clasificador supervisado TF-IDF + Regresion Logistica con un agente ReAct para evaluacion de CVs. El clasificador alcanza F1-macro de 0,799 y Accuracy de 0,830 sobre 13 categorias industriales; el agente evaluador logra una tasa de acierto del 86.7% en 15 casos estandarizados con una latencia promedio de 15.6 segundos por evaluacion.

Las mejoras mas directas para trabajo futuro incluyen: agregar un paso de refinamiento donde el agente revise su propio score antes de responder, incorporar la categoria predicha por el clasificador como contexto adicional para el agente, y ampliar el conjunto de evaluacion a 50 o 100 casos para obtener estimaciones estadisticamente mas robustas.

Referencias

- [1] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. arXiv preprint arXiv:2210.03629.
- [2] Chase, H. (2022). *LangChain* [Software]. <https://github.com/langchain-ai/langchain>
- [3] Groq Inc. (2024). *Groq API Documentation*. <https://console.groq.com/docs>
- [4] Singer-Vine, J. (2023). *pdfplumber* [Software]. <https://github.com/jsvine/pdfplumber>
- [5] Conor, A. (2024). *LinkedIn Job Postings Dataset* [Dataset]. Kaggle. <https://www.kaggle.com/datasets/arshkon/linkedin-job-postings>